

Trabajo Fin de Máster

DevMind

Sube tu proyecto. Pregúntale en español.

Una API que convierte cualquier proyecto de código
en algo que puedes preguntar con palabras normales.

Entrar en un proyecto ajeno cuesta días

Cuando un desarrollador llega a un proyecto que no ha escrito él, el conocimiento no está en un sitio. Está repartido.

- Cientos de carpetas y archivos
- Documentación incompleta o antigua
- Convenciones que nadie escribió
- Decisiones que solo recuerda quien se fue

proyecto/

```
├─ src/
|   ├─ application/
|   |   ├─ auth/
|   |   ├─ projects/
|   |   └─ ... 14 más
|   ├─ domain/
|   ├─ infrastructure/
|   ├─ transport/
|   └─ shared/
├─ tests/ (44 archivos)
├─ scripts/
└─ ... ¿dónde está el login?
```

“¿Dónde se gestiona el login aquí?”

Una pregunta de cinco segundos.

Una respuesta de dos horas.

¿Y por qué no se lo pregunto a ChatGPT?

1

No conoce tu código

Sabe programar, pero no ha visto nunca tu proyecto. Puede contarte cómo suele hacerse un login. No dónde está el tuyo.

2

Tu proyecto no cabe

Un proyecto real tiene miles de líneas. No entran en un chat. Y aunque entraran, el modelo se pierde entre tanto ruido.

3

Y si no sabe, se lo inventa

Sin el código delante, rellena huecos con lo que le parece razonable. En algo técnico, eso es peor que no responder.

DevMind no compite con esos modelos. Los usa — pero con el código correcto delante.

Leer el proyecto una vez. Responder muchas.

1

Una vez: organizar

Subes tu proyecto en un ZIP. DevMind lo lee entero, lo trocea en fragmentos pequeños y guarda cada uno con una especie de "etiqueta de significado".

Tarda un rato. Se hace una sola vez.

2

Cada vez: preguntar

Escribes una pregunta normal. DevMind busca solo los fragmentos que hablan de eso, se los pasa al modelo de IA y te da una respuesta.

Tarda segundos. Se hace siempre que quieras.

Como un bibliotecario

Nadie se lee la biblioteca entera para responder una pregunta. Va, coge tres libros, y lee las páginas que importan.

1

Primero, ordena la biblioteca

Cada libro a su estantería por tema. No alfabético: por de qué habla.

2

Luego, escucha la pregunta

"Busco algo sobre navegación del XVIII". No necesita el título exacto.

3

Va y trae solo lo relevante

Vuelve con cinco páginas. No con la biblioteca entera.

4

Y te lo lee en voz alta

Te resume lo que pone y te dice de qué libro lo ha sacado.

Trocea tu código y etiqueta cada fragmento por su significado.

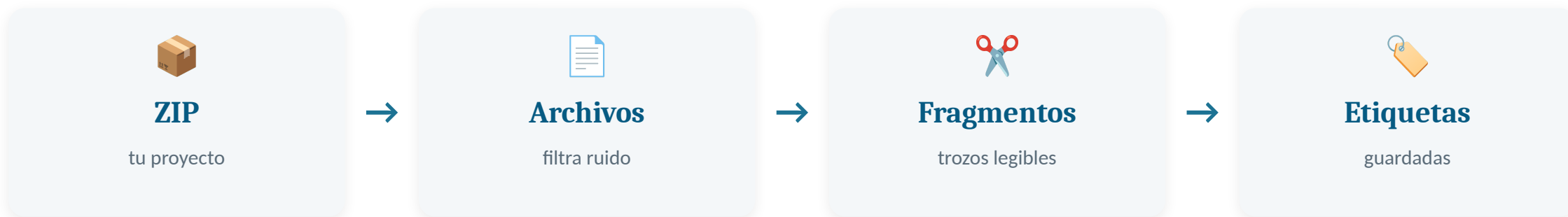
Preguntas en español. No hace falta acertar el nombre del archivo.

Recupera los 5 fragmentos de código más cercanos a tu pregunta.

La IA redacta la respuesta y cita los archivos y líneas exactas.

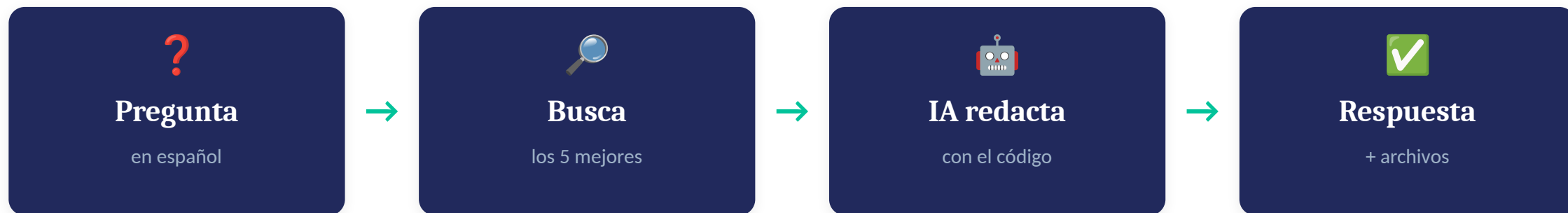
De un ZIP a una respuesta

UNA VEZ • al subir el proyecto



Se hace en segundo plano: tú no esperas.

CADA VEZ • al preguntar



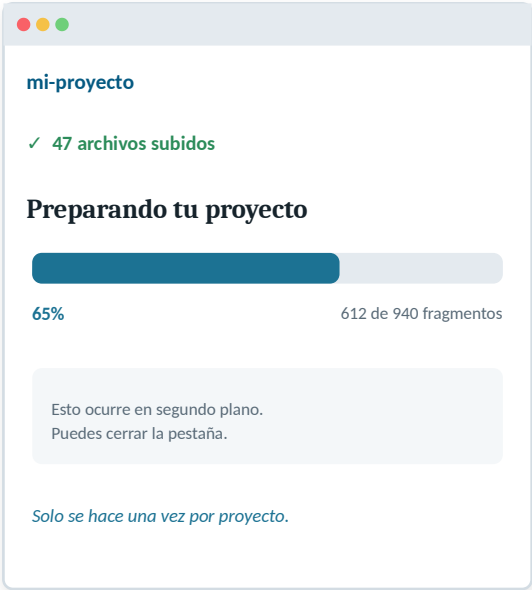
Dos o tres segundos.

Cuatro pantallas, de principio a fin

1 • Subir el ZIP



2 • Se organiza solo



3 • Preguntar



4 • Respuesta con fuentes



Invitado — para probar

Todo caduca a las 24 h y se borra: proyecto, archivos y preguntas. Al volver, empiezas de cero.



Registrado — para quedarte

Tus proyectos y todo tu historial de conversaciones se guardan y siguen ahí cuando vuelvas.

Esto es lo que devuelve

? ¿Cómo funciona el registro de usuarios?

RESPUESTA

El registro se gestiona en RegisterUserUseCase. El flujo comprueba primero si ya existe un usuario con ese email y, si existe, lanza un error de usuario duplicado.

Si no existe, cifra la contraseña con bcrypt, genera un identificador único y guarda el usuario en el repositorio. La contraseña original nunca se almacena: solo su hash.

La entrada llega por POST /auth/register, previa validación del body con Zod.

FUENTES USADAS

● **registerUserUseCase.ts**

líneas 13-37

● **authController.ts**

líneas 18-34

● **bcryptPasswordHasher.ts**

líneas 1-19

● **authRoutes.ts**

líneas 8-21

● **authSchemas.ts**

líneas 3-12

Puedes abrir cada archivo y comprobarlo.

La respuesta no la ha inventado: está construida con esos cinco fragmentos de código delante.

Una IA que sabe decir “no lo sé”

El mayor riesgo de estas herramientas es que respondan bien a todo — incluso a lo que no saben.

DevMind está diseñado para no hacerlo.



Responde con tu código delante

La respuesta no sale de lo que el modelo recuerde de internet. Sale de los fragmentos de TU proyecto que ha recuperado un segundo antes.



Cita siempre sus fuentes

Cada respuesta viene con la lista de archivos y líneas exactas que ha usado. Puedes ir y comprobarlo tú mismo.



Si no encuentra nada, lo dice

Hay un umbral de relevancia. Si ningún fragmento se parece lo bastante a la pregunta, DevMind contesta que no tiene información — en vez de inventar.

Seguridad y límites



Cada usuario solo ve lo suyo

El proyecto se busca siempre por proyecto + dueño a la vez. Con el id de otro no se llega a nada.



Contraseñas nunca en claro

Se guarda una huella irreversible, no la contraseña.



Modo invitado

Se puede probar sin registrarse. El invitado no deja historial y se limpia solo.

Límites conscientes

Tamaño máximo del ZIP, para que nadie tumbe el servidor

Límite de intentos de login, contra ataques por fuerza bruta

Se ignoran librerías, imágenes y binarios — solo código real

Por ahora es una API con frontend de prueba, no un producto cerrado

Tres capas que no se pisan

La decisión de diseño más importante del proyecto: separar el “qué hace” del “con qué está hecho”.

La puerta de entrada

Recibe las peticiones, comprueba quién eres y valida lo que envías. No decide nada.

Aquí vive Express, el login por token, los límites de peticiones.



La lógica del proyecto

Las reglas de negocio: registrar, subir, trocear, buscar, responder. El corazón.

No sabe si los datos están en PostgreSQL o en memoria. No sabe qué IA hay detrás.



Las herramientas

Lo concreto e intercambiable: la base de datos, el modelo de IA, el lector de ZIP.

Aquí vive PostgreSQL, Gemini, bcrypt. Cada uno enchufado por un conector.

¿Y esto para qué sirve?



Cambiar de IA o de base de datos toca una pieza, no el proyecto



Cada pieza se prueba sola, sin levantar nada más



Las flechas van en un solo sentido: el corazón no depende de nadie

No es un prototipo de una tarde

176

pruebas automáticas

escritas antes que el código

18

operaciones de negocio

cada una aislada y testeada

100

archivos de código

en 6 capas separadas

9

migraciones de BD

el esquema es reproducible

Por qué importa esto

El proyecto está construido en capas separadas: la lógica del negocio no sabe nada de PostgreSQL, ni de Google, ni de Express. Si mañana quiero cambiar el modelo de IA o la base de datos, toco una pieza — no el proyecto entero. Y puedo probar cada pieza por separado: de ahí salen esas 176 pruebas.

Dos decisiones que lo cambiaron todo

El problema

Subir un ZIP tardaba una eternidad

Subir y organizar era una sola operación. Con un proyecto grande, el usuario se quedaba mirando una pantalla congelada durante minutos.

La solución

Separé las dos cosas. Subir responde al instante; organizar ocurre en segundo plano y se puede consultar el progreso.

El problema

Respondía a todo, incluso sin saber

La búsqueda siempre devuelve lo “más cercano” a la pregunta. Pero más cercano no es relevante: si preguntabas por cocina, te daba código igual.

La solución

Añadí un umbral de relevancia. Si nada se parece lo bastante, DevMind admite que no lo sabe. Fue lo que más mejoró la confianza.

DevMind

Convierte un proyecto de código en algo que puedes preguntar.

Funciona hoy

- ✓ Registro, login y modo invitado
- ✓ Subida de proyectos en ZIP
- ✓ Organización automática en segundo plano
- ✓ Preguntas en lenguaje natural con fuentes
- ✓ Historial de conversación por proyecto

Con más tiempo

- Trocear por funciones, no por líneas
- Conectar directamente con GitHub
- Respuestas que llegan escribiéndose
- Medir la calidad de las respuestas

Gracias

El detalle técnico completo está en la documentación del repositorio.